



ÉCOLE NORMALE SUPÉRIEURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMED VI
UNIVERSITÉ HASSAN II DE CASABLANCA

DÉPARTEMENT MATHÉMATIQUES ET INFORMATIQUE

MÉMOIRE DE PROJET PROFESSIONNEL



Mizan

Plateforme web et mobile de bien-être, suivi académique et
accompagnement intelligent des étudiants

Réalisé par :

- Imgharn Noureddine
- Ed Deryouch Mohamed
- Elharmali Mousaab
- Charrafi Ayoub

Encadré par :

- Hamida Soufiane
- Karjoun Hasan

Année universitaire : 2025-2026

Dédicace

Nous dédions ce travail à nos familles pour leur soutien constant, à nos enseignants pour leur accompagnement pédagogique, ainsi qu'à toutes les personnes qui nous ont encouragés durant la réalisation de ce projet.

Remerciements

Nous tenons à remercier nos encadrants, Hamida Soufiane et Karjoun Hasan, pour leur accompagnement, leurs remarques et leur suivi durant la réalisation de ce projet. Nous remercions également l'École Normale Supérieure de l'Enseignement Technique de Mohammedia pour le cadre de formation et les ressources pédagogiques mises à notre disposition.

Nos remerciements vont aussi aux enseignants et aux camarades qui ont contribué, par leurs retours et leurs questions, à améliorer la réflexion fonctionnelle et technique autour de la plateforme Mizan.

Résumé

Mizan est une plateforme web et mobile destinée au suivi du bien-être étudiant et à l'organisation académique. Le projet répond à un besoin concret observé dans les parcours de formation : les étudiants doivent gérer simultanément les cours, examens, projets, objectifs personnels, fatigue, humeur et périodes de stress. Les outils classiques de planification traitent souvent uniquement les tâches, sans prendre en compte l'état émotionnel ni le contexte pédagogique.

La solution proposée combine un back-end FastAPI, une application web Next.js, une application mobile Expo React Native, une base PostgreSQL et des services d'intelligence artificielle. Elle permet aux étudiants de réaliser des check-ins matin et soir, suivre leurs objectifs, organiser leurs modes de travail, consulter leurs notifications et dialoguer avec un assistant intelligent. Une interface d'administration permet de gérer les institutions, classes, étudiants, contenus pédagogiques, ressources et indicateurs analytiques.

Mots-clés : bien-être étudiant, FastAPI, Next.js, React Native, PostgreSQL, intelligence artificielle, architecture cloud, analytique.

Abstract

Mizan is a web and mobile platform designed to support student wellbeing and academic organization. The project addresses a practical need: students must manage courses, exams, projects, personal goals, fatigue, mood and stress at the same time. Traditional planning tools usually focus on tasks only and do not fully consider emotional state or academic context.

The proposed solution combines a FastAPI back-end, a Next.js web application, an Expo React Native mobile application, a PostgreSQL database and artificial intelligence services. It allows students to complete morning and evening check-ins, track goals, organize work modes, receive notifications and interact with an intelligent assistant. The administration interface supports institutional management, class content, student provisioning, wellbeing resources and analytical dashboards.

Keywords: student wellbeing, FastAPI, Next.js, React Native, PostgreSQL, artificial intelligence, cloud architecture, analytics.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iii
Liste des Abréviations et Sigles	vii
Introduction Générale	1
Contexte général du projet	1
Problématique	1
Objectifs du projet	1
Intérêt du projet	2
Méthodologie adoptée	2
Moyens utilisés	2
1 Présentation du Cadre du Projet	3
1.1 Présentation de l'organisme d'accueil	3
1.2 Contexte du projet	3
1.3 Étude de l'existant	4
1.4 Critique de l'existant	5
1.5 Proposition de solution	5
2 Analyse des Besoins et Spécifications	5
2.1 Recueil des besoins	5
2.2 Identification des acteurs	6
2.3 Besoins fonctionnels	7
2.4 Besoins non fonctionnels	8
2.5 Spécifications fonctionnelles	9
2.6 Synthèse UML des scénarios	10
2.7 Cahier des charges	14
3 Conception de la Solution	15
3.1 Architecture générale	15
3.2 Choix technologiques	16
3.3 Conception fonctionnelle	17
3.4 Conception technique	18
3.5 Modélisation UML	19
3.6 Conception des interfaces	26
4 Réalisation et Implémentation	28
4.1 Environnement de développement	28
4.2 Architecture réalisée	28
4.3 Implémentation des fonctionnalités	29
4.4 Intégration avec la base de données	30

4.5	Sécurité et contrôle d'accès	30
4.6	Plan de validation et qualité	31
4.7	Préparation du déploiement et exploitation	32
4.8	Difficultés rencontrées	33
Conclusion Générale		33
	Bilan du projet	33
	Réponse à la problématique	33
	Objectifs atteints	34
	Limites du travail réalisé	34
	Perspectives d'amélioration	34
	Apport personnel et professionnel	35
Annexe A – Glossaire Technique		36
Bibliographie / Webographie		36

Liste des figures

1	Diagramme de cas d'utilisation de Mizan	11
2	Diagramme de contexte système C4 niveau 1	15
3	Diagramme de conteneurs C4 niveau 2	16
4	Diagramme de composants du back-end C4 niveau 3	17
5	Diagramme de code C4 niveau 4 du module check-in	18
6	Diagramme de séquence du check-in avec analyse intelligente	21
7	Diagramme de classes détaillé du domaine Mizan	23
8	Diagramme de déploiement AWS	25

Liste des tableaux

1	Liste des abréviations et sigles	vii
2	Comparaison synthétique de l'existant	4
3	Acteurs humains et systèmes externes	6
4	Données manipulées par acteur	7
5	Besoins fonctionnels de Mizan	7
6	Priorisation fonctionnelle	8
7	Critères non fonctionnels	9
8	Règles de gestion principales	9
9	Scénarios d'utilisation détaillés	10
10	Traçabilité entre besoins et modules	10
11	Spécification des cas d'utilisation de l'espace étudiant	12
12	Spécification des cas d'utilisation d'accompagnement étudiant	13
13	Spécification des cas d'utilisation administrateur	14
14	Choix technologiques et justification	17
15	Flux fonctionnels détaillés	18
16	Principes de conception appliqués	19
17	Rôle des diagrammes UML dans Mizan	20
18	Lecture fonctionnelle du diagramme de classes	24
19	Rôle des composants de déploiement	25
20	Correspondance entre profils et interfaces	26
21	Parcours utilisateur et exigences d'interface	27
22	Organisation logique des écrans	27
23	Environnement de développement	28
24	Correspondance entre conception et réalisation	29
25	Utilisation des données principales	30
26	Stratégie de tests et validation	31
27	Matrice de validation fonctionnelle	32
28	Éléments de déploiement et exploitation	32
29	Perspectives d'amélioration de Mizan	35
30	Glossaire technique	36

Liste des Abréviations et Sigles

Sigle	Signification
API	Interface de programmation applicative.
AWS	Amazon Web Services, plateforme cloud ciblée pour l'architecture de déploiement.
CI/CD	Intégration continue et déploiement continu.
CORS	Cross-Origin Resource Sharing, mécanisme de contrôle des origines web autorisées.
CRUD	Create, Read, Update, Delete : opérations de base sur les données.
ECR	Elastic Container Registry, registre Docker d'AWS.
ECS	Elastic Container Service, service d'orchestration de conteneurs AWS.
JWT	JSON Web Token, format de jeton utilisé pour l'authentification.
ORM	Object-Relational Mapping, technique de correspondance entre objets et base relationnelle.
PWA	Progressive Web App.
RDS	Relational Database Service, service AWS de base de données managée.
SGBD	Système de gestion de base de données.
UML	Unified Modeling Language, langage de modélisation orienté objet.

Table 1: Liste des abréviations et sigles

Introduction Générale

Contexte général du projet

La réussite académique ne dépend pas uniquement de la disponibilité des cours ou de la capacité à créer une liste de tâches. Elle dépend aussi de facteurs personnels : sommeil, humeur, charge mentale, échéances rapprochées, qualité de l'organisation et capacité à demander de l'aide au bon moment. Dans les filières informatiques et scientifiques, ces facteurs deviennent visibles pendant les périodes d'examens, de projets et de soutenances, où la charge de travail peut augmenter rapidement.

Mizan est né de cette observation. Le projet cherche à créer un environnement numérique qui accompagne l'étudiant dans son quotidien au lieu de se limiter à une gestion statique de tâches. La plateforme collecte des signaux simples, comme l'humeur, le sommeil, les objectifs, les cours, les examens et les projets, puis les transforme en recommandations exploitables.

Le choix de travailler sur une plateforme web et mobile s'explique par les habitudes d'utilisation des étudiants. Le web reste adapté aux tâches de consultation, de gestion et d'administration, tandis que le mobile est plus approprié aux interactions rapides comme les check-ins, les notifications et les rappels. Cette complémentarité permet de couvrir les usages ponctuels et les usages plus structurés.

Problématique

Les étudiants utilisent souvent plusieurs outils séparés pour gérer leur emploi du temps, leurs tâches, leurs projets et leur état personnel. Cette fragmentation rend difficile une vision globale de la journée. Un étudiant peut connaître ses examens sans relier cette information à son niveau de fatigue, ou suivre ses tâches sans prendre en compte son humeur et sa capacité réelle de concentration.

La problématique traitée par Mizan peut être formulée ainsi :

Problématique

Comment concevoir une plateforme numérique capable de centraliser le contexte académique et les signaux de bien-être afin d'aider l'étudiant à mieux organiser son travail et permettre à l'administration de suivre des indicateurs utiles sans alourdir les processus existants ?

Objectifs du projet

Les objectifs principaux sont :

- centraliser les données académiques utiles à l'étudiant : emploi du temps, examens, projets et objectifs ;
- suivre le bien-être par des check-ins réguliers et des historiques exploitables ;
- proposer un assistant intelligent capable de produire des conseils contextualisés ;
- fournir aux administrateurs une vue claire sur les classes, étudiants et indicateurs de risque ;

- sécuriser l'accès par authentification, rôles et règles de production ;
- préparer une architecture déployable sur une infrastructure cloud.

Intérêt du projet

L'intérêt de Mizan est double. Pour l'étudiant, la plateforme réduit la dispersion des outils et propose un accompagnement plus régulier. Pour l'administration, elle donne une vision structurée des classes, des contenus académiques et des tendances générales de bien-être. Sur le plan technique, le projet met en pratique une architecture moderne couvrant API, web, mobile, base de données, intelligence artificielle et préparation du déploiement.

Méthodologie adoptée

Le projet a été mené selon une démarche incrémentale. Les besoins ont d'abord été identifiés, puis les modules prioritaires ont été réalisés progressivement : authentification, structure institutionnelle, profils étudiants, check-ins, objectifs, modes, assistant, notifications et tableaux de bord.

La méthodologie adoptée repose également sur une séparation claire entre l'analyse fonctionnelle et la réalisation technique. Les besoins ont été traduits en modules, puis chaque module a été associé à des routes API, des schémas de validation, des modèles de données et des services métier. Cette démarche facilite la traçabilité entre le besoin initial et le code réalisé.

Moyens utilisés

Les moyens utilisés sont :

- **Back-end** : Python 3.12, FastAPI, SQLAlchemy, Alembic et PostgreSQL ;
- **Front-end** : Next.js, React, TypeScript, Tailwind CSS et Axios ;
- **Mobile** : Expo React Native et TypeScript ;
- **IA et médias** : Mistral pour l'assistant et Cloudinary pour les photos ;
- **Préparation du déploiement** : Docker, Terraform, GitHub Actions, AWS ECS, RDS, ECR, ALB et CloudFront.

Ces moyens ont été choisis pour construire une solution cohérente de bout en bout. FastAPI fournit une API performante et lisible. Next.js permet de créer une interface web moderne. Expo simplifie le développement mobile. PostgreSQL assure une persistance relationnelle robuste. Enfin, Docker, Terraform et GitHub Actions structurent la préparation du déploiement.

1 Présentation du Cadre du Projet

1.1 Présentation de l'organisme d'accueil

Historique

Le projet est réalisé dans un cadre académique au sein du Département Mathématiques et Informatique de l'École Normale Supérieure de l'Enseignement Technique de Mohammedia. L'établissement accompagne la formation d'étudiants dans des domaines liés à l'informatique, à l'ingénierie logicielle et aux systèmes d'information.

Activités principales

Les activités principales concernent la formation, l'encadrement pédagogique, la réalisation de projets académiques, l'évaluation des compétences techniques et la préparation des étudiants aux besoins du marché numérique.

Dans ce contexte, les projets applicatifs jouent un rôle important, car ils permettent de relier les connaissances théoriques à des cas réels : analyse des besoins, modélisation, développement, tests, documentation et déploiement. Mizan s'inscrit dans cette logique en mobilisant plusieurs compétences du génie logiciel.

Organisation générale

L'organisation générale repose sur des départements, des filières, des promotions, des classes, des enseignants et des encadrants. Cette structure se reflète dans Mizan par les entités école, filière, promotion et classe.

Services concernés

Les services concernés par le projet sont principalement les services pédagogiques et administratifs. Les étudiants utilisent l'application pour leur suivi personnel, tandis que les administrateurs gèrent les classes, les ressources et les indicateurs.

Le service pédagogique est concerné par la gestion des emplois du temps, examens et projets. Le service administratif est concerné par la structuration des écoles, filières, promotions et classes. Les étudiants, quant à eux, sont concernés par l'usage quotidien : organisation, suivi, consultation et interaction avec l'assistant.

1.2 Contexte du projet

Origine du besoin

Le besoin provient de la difficulté à suivre simultanément les contraintes académiques et l'état personnel de l'étudiant. Les périodes d'examens, les projets de groupe et les charges de cours peuvent créer une pression importante. Une plateforme qui relie contexte académique et bien-être peut aider à mieux organiser les priorités.

Situation actuelle

Dans une situation classique, les étudiants utilisent plusieurs outils distincts : calendrier, applications de notes, messagerie, documents partagés et parfois applications de tâches. Les

administrateurs disposent de données académiques, mais rarement d'indicateurs simples reliant ces données au suivi quotidien des étudiants.

Enjeux métier et techniques

Les enjeux métier sont l'accompagnement, la prévention de la surcharge et l'amélioration de l'organisation. Les enjeux techniques sont la sécurité des données, la modularité de l'architecture, la disponibilité du service, l'intégration de l'intelligence artificielle et la capacité de déploiement.

1.3 Étude de l'existant

Solutions existantes

Les solutions existantes se divisent généralement en applications de tâches, plateformes pédagogiques, calendriers, applications de bien-être et outils d'analyse. Chacune couvre une partie du besoin, mais rarement l'ensemble du parcours étudiant.

Famille d'outils	Apport principal	Limite observée
Calendriers	Planification des cours et échéances.	Peu de prise en compte de l'humeur, de la fatigue ou de la charge mentale.
Applications de tâches	Organisation personnelle et priorisation simple.	Faible lien avec les contenus académiques réels.
Plateformes pédagogiques	Centralisation des cours, devoirs et documents.	Peu orientées bien-être et accompagnement quotidien.
Applications de bien-être	Suivi d'habitudes, humeur ou relaxation.	Généralement séparées du parcours académique.
Tableaux de bord analytiques	Visualisation d'indicateurs et tendances.	Peu accessibles à l'étudiant dans son usage quotidien.

Table 2: Comparaison synthétique de l'existant

Fonctionnement actuel

Le fonctionnement actuel est souvent manuel ou dispersé. Les données académiques sont gérées séparément des données personnelles. Les rappels et recommandations ne sont pas toujours contextualisés par les échéances réelles de l'étudiant.

Forces

Les outils existants sont simples, spécialisés et parfois très efficaces dans leur domaine : calendrier, liste de tâches, suivi d'habitudes ou plateforme pédagogique.

Limites

Leur limite principale est le manque d'intégration. Un outil de tâches ne sait pas toujours qu'un examen est prévu demain, et une application de bien-être ne connaît pas forcément

la charge de projets de l'étudiant.

1.4 Critique de l'existant

Problèmes rencontrés

Les problèmes rencontrés sont la duplication des informations, l'absence de vision globale, la difficulté à prioriser et le manque de recommandations adaptées au contexte réel.

Insuffisances

Les insuffisances concernent la personnalisation, la prise en compte du bien-être, la centralisation des données et la visibilité administrative sur les tendances générales.

Besoins d'amélioration

Il est nécessaire de proposer un outil unifié, accessible depuis le web et le mobile, capable de gérer les données académiques, les check-ins, les objectifs, les notifications et les recommandations.

1.5 Proposition de solution

Idee générale de la solution

Mizan propose une solution intégrée qui relie les dimensions académiques, personnelles et organisationnelles. L'étudiant peut déclarer son état du matin ou du soir, consulter son tableau de bord, suivre ses objectifs, gérer ses modes de travail, recevoir des notifications et dialoguer avec un agent d'accompagnement.

Apports attendus

Les apports attendus sont une meilleure organisation quotidienne, une centralisation des informations utiles, une aide à la priorisation, des indicateurs administratifs plus lisibles et une architecture technique prête à évoluer.

La solution attendue doit aussi favoriser une utilisation régulière. Pour cela, elle doit rester simple, rapide et rassurante. L'étudiant doit comprendre immédiatement ce que la plateforme lui apporte : une vue de sa journée, un suivi de ses objectifs, un rappel pertinent ou un conseil contextualisé. L'administrateur doit, de son côté, gagner du temps dans la gestion et la consultation des informations.

2 Analyse des Besoins et Spécifications

2.1 Recueil des besoins

Besoins métiers

Les besoins métiers consistent à mieux suivre les étudiants, organiser les contenus de classe, fournir des ressources de soutien, repérer les situations de surcharge et améliorer la communication des rappels importants.

Besoins utilisateurs

Les étudiants ont besoin d'un tableau de bord clair, de check-ins rapides, d'objectifs suivis, de rappels utiles et d'un assistant qui tient compte de leur contexte. Les administrateurs ont besoin d'outils de gestion simples et d'indicateurs synthétiques.

Besoins techniques

Les besoins techniques couvrent la sécurité, la séparation des rôles, la persistance fiable des données, la validation des fichiers, les tests, l'intégration continue et un déploiement reproductible.

Le recueil des besoins a également mis en évidence la nécessité d'un système simple à utiliser au quotidien. L'étudiant ne doit pas passer beaucoup de temps à renseigner son état : un check-in doit rester court, compréhensible et accessible depuis le téléphone. De son côté, l'administrateur doit pouvoir alimenter rapidement les données institutionnelles et académiques sans manipulations techniques complexes.

2.2 Identification des acteurs

Acteur	Rôle
Étudiant	Réalise les check-ins, suit ses objectifs, consulte ses tâches, utilise l'assistant et reçoit les notifications.
Administrateur d'école	Gère l'école, les filières, promotions, classes, étudiants, contenus académiques et ressources.
Administrateur global	Vérifie les écoles, supervise la plateforme et consulte des statistiques globales.
Assistant IA	Analyse le contexte disponible et génère des recommandations, plans d'action ou notifications.
Systèmes externes	Mistral pour l'IA, Cloudinary pour les médias, AWS pour le déploiement et GitHub Actions pour la CI/CD.

Table 3: Acteurs humains et systèmes externes

Responsabilités détaillées des acteurs

L'étudiant est l'acteur central de la plateforme. Il renseigne ses check-ins, consulte son tableau de bord, suit ses objectifs, organise ses tâches et interagit avec l'assistant. L'administrateur d'école intervient sur la qualité des données : création des classes, import des étudiants, ajout des examens, projets et ressources. L'administrateur global assure un rôle de supervision et de validation. Les systèmes externes complètent la plateforme sans remplacer le cœur applicatif : l'IA produit des recommandations, Cloudinary stocke les photos et AWS représente l'environnement cloud cible.

Acteur	Données manipulées	Résultat attendu
Étudiant	Check-ins, objectifs, tâches, modes, profil.	Meilleure organisation personnelle et suivi régulier.
Administrateur d'école	Classes, étudiants, emplois du temps, examens, projets, ressources.	Données pédagogiques fiables et indicateurs exploitables.
Administrateur global	Écoles, vérification, statistiques globales.	Supervision de la plateforme et cohérence institutionnelle.
Assistant IA	Contexte étudiant, historique, objectifs, échéances.	Recommandation, résumé ou notification adaptée.

Table 4: Données manipulées par acteur

2.3 Besoins fonctionnels

Module	Fonctionnalités attendues
Gestion des comptes	Activation, connexion, renouvellement de jeton, changement de mot de passe et contrôle des rôles.
Gestion des données	Création et mise à jour des écoles, filières, promotions, classes, étudiants, contenus et ressources.
Recherche et consultation	Consultation du tableau de bord, de l'historique, des objectifs, des ressources et des indicateurs.
Opérations CRUD	Ajout, lecture, modification et suppression des données principales selon les droits.
Notifications	Livraison REST et temps réel pour les rappels et actions autonomes.
Rapports et statistiques	Tableaux de bord étudiant et administrateur : humeur, modes, indicateurs de classe et tendances.

Table 5: Besoins fonctionnels de Mizan

Priorisation des fonctionnalités

Les fonctionnalités prioritaires sont celles qui permettent au système d'être utilisable de bout en bout : authentification, gestion des étudiants, check-ins, tableau de bord, objectifs, contenus académiques et sécurité. Les fonctionnalités importantes mais secondaires concernent l'enrichissement de l'assistant, les notifications avancées, les ressources de bien-être et les statistiques détaillées. Cette priorisation permet de livrer une première version cohérente avant d'élargir le périmètre.

Priorité	Fonctionnalités	Justification
Haute	Authentification, check-ins, tableau de bord, gestion étudiants, contenus académiques.	Ces modules constituent le parcours minimal utilisable.
Moyenne	Assistant, notifications, ressources, statistiques administrateur.	Ces modules enrichissent l'accompagnement et l'analyse.
Évolutive	Notifications mobiles natives, export avancé, expérimentation multi-établissements.	Ces modules préparent une version plus complète.

Table 6: Priorisation fonctionnelle

2.4 Besoins non fonctionnels

- **Sécurité** : JWT, hachage des mots de passe, rôles et validation de configuration en production.
- **Performance** : API asynchrone, base PostgreSQL et séparation des composants.
- **Fiabilité et disponibilité** : endpoint de santé, conteneurisation et déploiement cloud.
- **Maintenabilité et évolutivité** : architecture par routes, services, schémas et modèles.
- **Ergonomie** : interface web responsive et application mobile adaptée aux check-ins rapides.

Ces exigences non fonctionnelles sont essentielles, car la plateforme manipule des données personnelles et des données liées au bien-être. La performance garantit une expérience fluide, surtout lors de la consultation du tableau de bord. La maintenabilité facilite l'ajout de nouveaux modules. L'ergonomie est également critique : si le check-in est trop long ou trop complexe, l'étudiant risque de ne pas l'utiliser régulièrement.

Exigence	Critère attendu	Moyen prévu
Sécurité	Accès limité selon le rôle.	JWT, dépendances FastAPI et scoping administrateur.
Performance	Réponses rapides pour les pages fréquentes.	API asynchrone et requêtes ciblées.
Fiabilité	Application testable et déployable.	CI/CD, endpoint de santé et conteneurs.
Maintenabilité	Ajout de module sans réécrire l'existant.	Séparation routes, schémas, services et modèles.

Exigence	Critère attendu	Moyen prévu
Ergonomie	Parcours court pour les check-ins.	Interface claire, mobile et web.

Table 7: Critères non fonctionnels

2.5 Spécifications fonctionnelles

Règles de gestion

Règle	Description
Séparation des rôles	Un étudiant accède à ses propres données, tandis qu'un administrateur agit dans le périmètre de son établissement.
Activation contrôlée	Un compte étudiant doit être activé avant l'utilisation complète de l'application.
Contexte académique	Les examens, projets et emplois du temps enrichissent les recommandations.
Check-in quotidien	Les réponses du matin et du soir alimentent l'historique, le tableau de bord et l'assistant.
Mode de secours IA	Si la clé du fournisseur IA est absente, l'application fournit une réponse locale minimale.
Protection des fichiers	Les fichiers image, audio et CSV sont validés avant traitement.

Table 8: Règles de gestion principales

Scénarios d'utilisation

Les scénarios principaux sont : connexion étudiant, check-in matinal, check-in du soir, consultation du tableau de bord, création d'objectif, démarrage d'un mode de travail, import d'étudiants, création d'examen, consultation analytique et déclenchement de notification.

Scénario	Description détaillée
Check-in matinal	L'étudiant se connecte, saisit son humeur, son sommeil et ses priorités. Le système enregistre les réponses, met à jour l'historique et peut générer un plan d'action court.
Check-in du soir	L'étudiant indique si son plan a été réalisé, ajoute des notes et consulte un résumé de la journée.
Gestion académique	L'administrateur ajoute les emplois du temps, examens et projets afin d'enrichir le contexte étudiant.
Assistant intelligent	Le système construit le contexte à partir des données académiques, des objectifs et des check-ins, puis produit une recommandation adaptée.

Scénario	Description détaillée
Statistiques	L'administrateur consulte des indicateurs agrégés pour comprendre les tendances générales d'une classe ou d'un établissement.

Table 9: Scénarios d'utilisation détaillés

Contraintes fonctionnelles

Les contraintes fonctionnelles concernent la cohérence des rôles, la validation des dates, l'unicité des comptes, la limitation des fichiers, la cohérence des données institutionnelles et la disponibilité du mode de secours lorsque l'IA externe n'est pas configurée.

Les données doivent rester cohérentes entre les modules. Un étudiant doit être rattaché à une classe existante. Un examen ou un projet doit être associé à un étudiant ou à un groupe cohérent. Les notifications ne doivent pas être envoyées sans contexte suffisant, afin d'éviter les rappels inutiles. Enfin, les recommandations de l'assistant doivent rester informatives et ne doivent pas être présentées comme des décisions médicales.

Traçabilité besoins-modules

Besoin	Module concerné	Livrable associé
Suivre le bien-être quotidien	Check-ins	Formulaires matin/soir et historique.
Organiser les priorités	Objectifs, tâches, modes	Pages objectifs, tâches et sessions de travail.
Contextualiser les conseils	Agent, contexte étudiant	Service de contexte et assistant intelligent.
Gérer les données académiques	Institution, class-content	Routes admin et imports CSV.
Informier l'étudiant	Notifications	API notifications et écoute temps réel.
Superviser les tendances	Analytics	Tableaux de bord étudiant et administrateur.

Table 10: Traçabilité entre besoins et modules

2.6 Synthèse UML des scénarios

Cette synthèse complète les spécifications par une vue UML centrale : le diagramme de cas d'utilisation, qui résume les interactions principales entre les acteurs et la plateforme. Ce diagramme assure une meilleure transition entre l'analyse des besoins et la conception technique.

Diagramme de cas d'utilisation

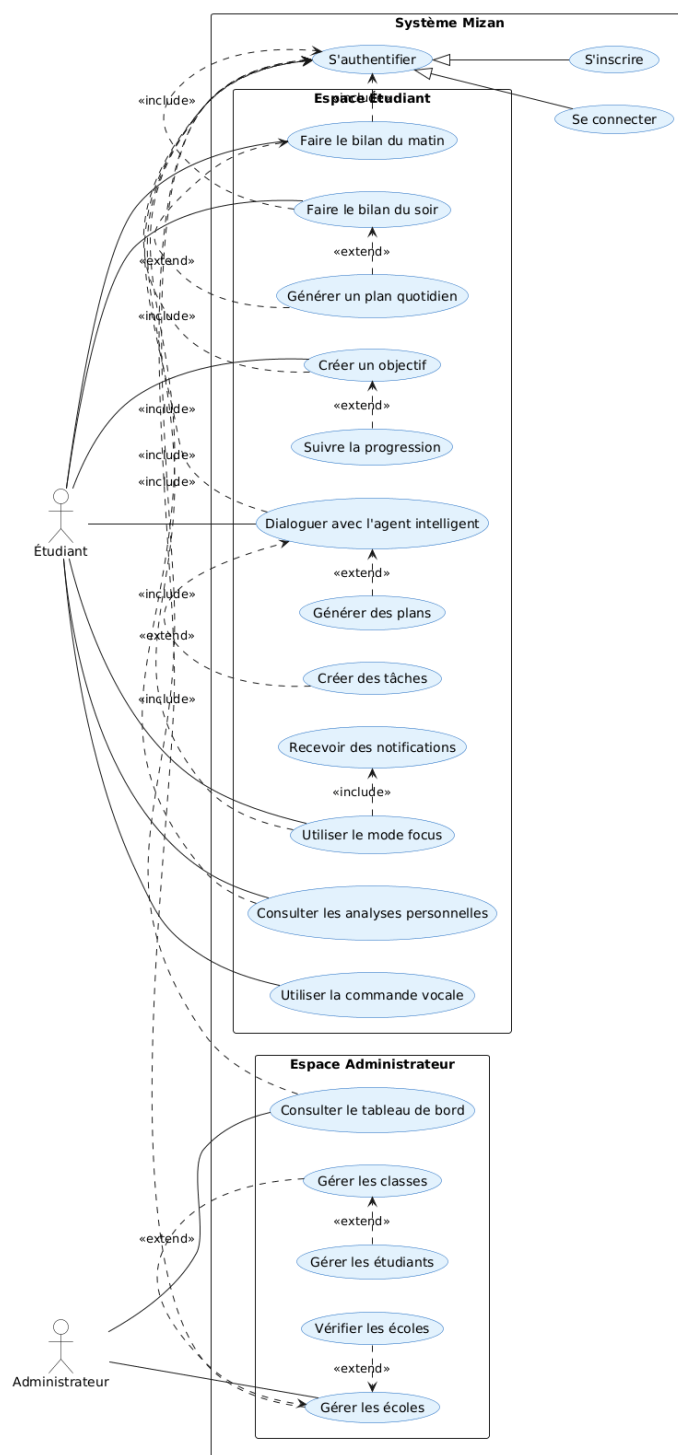


Figure 1: Diagramme de cas d'utilisation de Mizan

La spécification suivante décrit les cas d'utilisation visibles dans le diagramme. Elle précise, pour chaque fonction importante, l'acteur concerné, la condition d'accès, le déroulement nominal et le résultat attendu. Cette description permet de transformer le diagramme en exigences fonctionnelles exploitables pendant la conception.

Cas d'utilisation	Acteur	Précondition	Scénario nominal et postcondition
S'inscrire	Étudiant	L'étudiant ne possède pas encore de compte actif.	Il renseigne ses informations, valide le formulaire et obtient un compte utilisable après contrôle des données.
Se connecter / s'authentifier	Étudiant, Administrateur	Le compte existe et les identifiants sont disponibles.	L'utilisateur saisit ses identifiants, le système vérifie l'accès et ouvre l'espace correspondant à son rôle.
Faire le bilan du matin	Étudiant	L'étudiant est authentifié.	Il saisit son humeur, son sommeil et ses priorités. Le système enregistre le bilan et prépare les données de suivi de la journée.
Faire le bilan du soir	Étudiant	L'étudiant est authentifié et peut avoir déjà réalisé un bilan du matin.	Il indique l'état de sa journée, les tâches réalisées et les difficultés rencontrées. Le système complète l'historique quotidien.
Générer un plan quotidien	Étudiant, Agent intelligent	Des informations de contexte sont disponibles.	Le système analyse les bilans, objectifs, tâches et échéances afin de proposer un plan adapté à la journée.
Créer un objectif	Étudiant	L'étudiant est connecté à son espace.	Il ajoute un objectif avec un titre, une description et éventuellement une priorité. L'objectif devient consultable et modifiable.
Suivre la progression	Étudiant	Des objectifs, tâches ou check-ins existent.	Le système affiche l'évolution des objectifs, l'état des tâches et les indicateurs personnels.

Table 11: Spécification des cas d'utilisation de l'espace étudiant

Cas d'utilisation	Acteur	Précondition	Scénario nominal et postcondition
Dialoguer avec l'agent intelligent	Étudiant	L'étudiant est authentifié et dispose d'un contexte minimal.	Il pose une question. Le système prépare le contexte, interroge l'agent ou le fallback, puis retourne une réponse personnalisée.
Générer des plans	Étudiant, Agent intelligent	Les objectifs, tâches ou bilans peuvent être analysés.	L'agent propose des étapes concrètes pour organiser le travail, réduire la surcharge ou préparer une échéance.
Créer des tâches	Étudiant	L'étudiant est connecté.	Il crée une tâche liée à un objectif, un cours ou une priorité personnelle. La tâche est ajoutée au suivi.
Recevoir des notifications	Étudiant	Une action ou une condition de rappel est détectée.	Le système envoie un rappel ou une alerte liée aux objectifs, bilans, tâches ou recommandations.
Utiliser le mode focus	Étudiant	L'étudiant souhaite lancer une session de concentration.	Il démarre un mode de travail. Le système suit la session et peut l'associer à une tâche ou un objectif.
Consulter les analyses personnelles	Étudiant	Des données de suivi existent.	Le système présente des indicateurs sur l'humeur, le sommeil, la progression et les tendances personnelles.

Table 12: Spécification des cas d'utilisation d'accompagnement étudiant

Cas d'utilisation	Acteur	Précondition	Scénario nominal et postcondition
Consulter le tableau de bord	Administrateur	L'administrateur est authentifié.	Il accède aux indicateurs de suivi et obtient une vue synthétique des données de l'établissement.

Cas d'utilisation	Acteur	Précondition	Scénario nominal et postcondition
Gérer les classes	Administrateur	Une école ou une structure académique existe.	Il crée, modifie ou consulte les classes afin d'organiser les étudiants.
Gérer les étudiants	Administrateur	Les classes sont disponibles.	Il ajoute, importe, modifie ou consulte les profils étudiants rattachés à l'établissement.
Vérifier les écoles	Administrateur	Des écoles sont en attente de validation.	Il consulte les demandes, vérifie les informations et décide de l'acceptation ou du rejet.
Gérer les écoles	Administrateur	L'administrateur dispose des droits nécessaires.	Il administre les écoles, leurs informations et leur état de validation.

Table 13: Spécification des cas d'utilisation administrateur

Dans ce diagramme, la relation **include** signifie qu'un cas d'utilisation dépend d'un comportement obligatoire. L'authentification est donc incluse dans la majorité des fonctionnalités protégées, car l'étudiant ou l'administrateur doit être identifié avant d'accéder aux données. La relation **extend** indique un comportement optionnel ou conditionnel : par exemple, le bilan du soir complète le suivi quotidien, la génération de plans enrichit l'utilisation de l'agent intelligent, et les notifications peuvent être déclenchées après certaines actions ou certains changements d'état.

2.7 Cahier des charges

Objectifs détaillés

Le cahier des charges impose une plateforme multi-interfaces, sécurisée, modulaire, capable de gérer les étudiants, les contenus académiques, les check-ins, les objectifs, les ressources, l'assistant et les tableaux de bord.

- garantir un accès sécurisé aux espaces étudiant et administrateur ;
- permettre la saisie rapide des check-ins depuis le web ou le mobile ;
- fournir une vision synthétique de la journée de l'étudiant ;
- offrir aux administrateurs des outils simples de gestion et d'import ;
- intégrer un assistant capable de fonctionner même en mode dégradé ;

- préparer un déploiement cloud reproductible.

Contraintes

Les contraintes techniques incluent FastAPI, Next.js, React Native, PostgreSQL, Docker et CI/CD. Les contraintes logicielles portent sur la validation des données et la sécurité. Les contraintes temporelles imposent une progression par modules prioritaires.

Critères d'acceptation

Une première version de Mizan est considérée comme acceptable si l'étudiant peut se connecter, réaliser un check-in, consulter son tableau de bord et suivre au moins un objectif. L'administrateur doit pouvoir gérer une structure institutionnelle, ajouter des étudiants et créer des contenus académiques. Le système doit refuser les accès non autorisés, valider les données reçues, conserver les historiques et rester utilisable lorsque le fournisseur IA n'est pas configuré.

Les critères d'acceptation incluent aussi la lisibilité de l'interface, la cohérence des messages d'erreur, la possibilité d'exécuter les tests automatiquement et la capacité à produire un build front-end sans erreur. Le scénario minimal attendu est : création d'un étudiant, connexion, check-in, consultation du tableau de bord, interaction avec l'assistant et consultation d'un indicateur administrateur.

3 Conception de la Solution

3.1 Architecture générale

3.1.1 Diagramme de contexte système C4 – Niveau 1

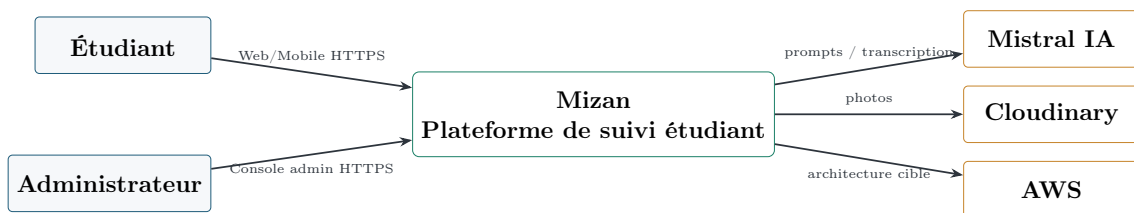


Figure 2: Diagramme de contexte système C4 niveau 1

3.1.2 Diagramme de conteneurs C4 – Niveau 2

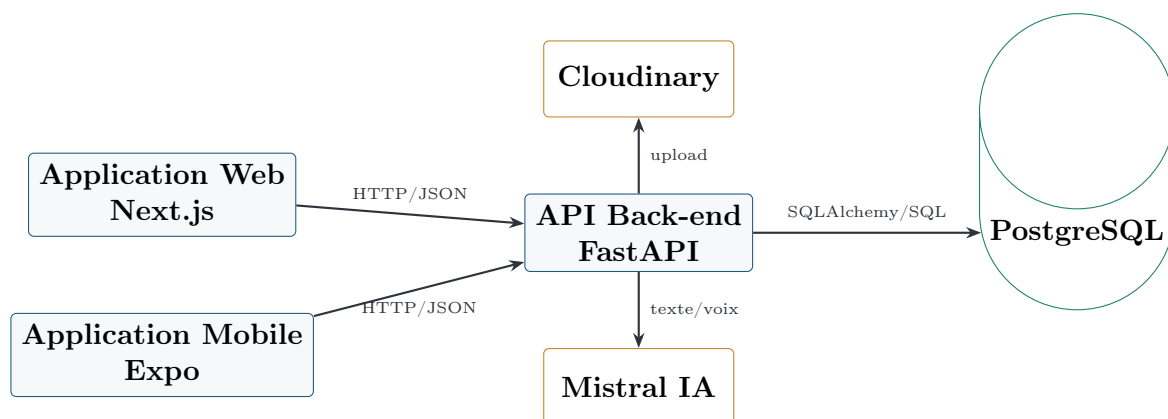


Figure 3: Diagramme de conteneurs C4 niveau 2

3.1.3 Architecture matérielle cible

L'architecture matérielle cible repose sur AWS. Le front-end et le back-end sont prévus sous forme de conteneurs ECS Fargate. La base PostgreSQL est prévue dans RDS. CloudFront et l'Application Load Balancer constituent les points d'entrée prévus pour l'exposition publique.

3.1.4 Architecture réseau cible

Le trafic utilisateur est prévu en HTTPS vers CloudFront puis vers l'Application Load Balancer. Les routes API sont redirigées vers le service back-end, tandis que les autres routes servent l'application front-end. La base RDS reste privée et accessible uniquement par le back-end.

3.2 Choix technologiques

Domaine	Choix	Justification
Langage back-end	Python	Écosystème riche, lisibilité et intégration rapide avec FastAPI.
Framework API	FastAPI	Performance, validation Pydantic et documentation automatique.
Front-end web	Next.js / React	Architecture moderne, routage clair et TypeScript.
Mobile	Expo React Native	Développement mobile rapide et multi-plateforme.
SGBD	PostgreSQL	Robustesse, relations fortes et compatibilité SQLAlchemy.
Déploiement	Docker, Terraform, AWS	Reproductibilité, infrastructure versionnée et cloud managé.

Domaine	Choix	Justification
---------	-------	---------------

Table 14: Choix technologiques et justification

3.3 Conception fonctionnelle

3.3.1 Diagramme de composants C4 – Niveau 3

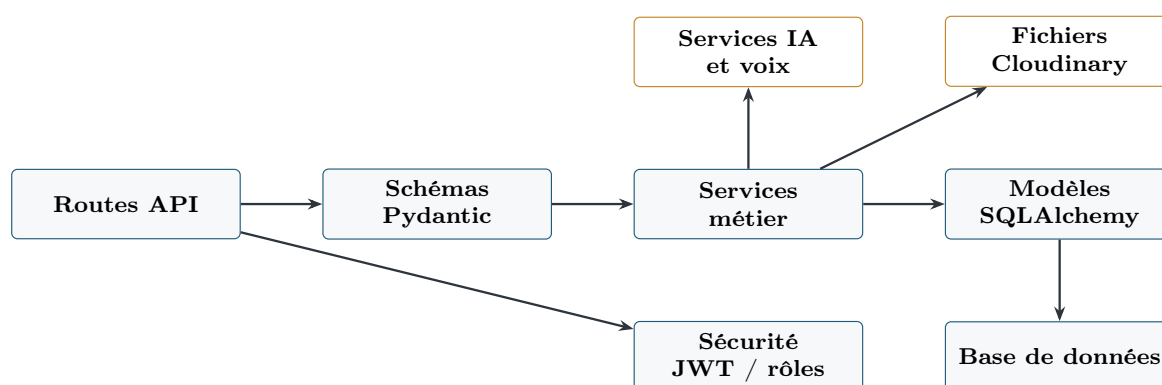


Figure 4: Diagramme de composants du back-end C4 niveau 3

3.3.2 Répartition et rôle de chaque module

Le dossier `routes/` expose les points d'accès HTTP. Le dossier `schemas/` valide les entrées et sorties. Le dossier `models/` représente les tables SQLAlchemy. Le dossier `services/` contient les règles métier. Le dossier `core/` centralise la configuration, la sécurité et la connexion à la base.

3.3.3 Flux de circulation des données

Les données saisies par l'utilisateur passent du front-end vers l'API. L'API valide la requête, applique les règles métier, persiste les données dans PostgreSQL et retourne une réponse structurée. Pour les fonctionnalités intelligentes, l'API construit un contexte et interroge le fournisseur IA.

Le flux de données est organisé de manière à conserver une séparation claire entre l'affichage, la validation, la logique métier et la persistance. Cette organisation limite les effets de bord : une modification de l'interface ne doit pas modifier directement les règles métier, et une évolution de la base de données doit passer par les modèles et les migrations. Les échanges entre les applications clientes et l'API utilisent principalement JSON sur HTTP, ce qui facilite l'intégration web et mobile.

Flux	Étapes principales	Résultat attendu
Check-in étudiant	Saisie mobile ou web, validation Pydantic, sauvegarde PostgreSQL, mise à jour de l'historique.	État quotidien enregistré et exploitable par le tableau de bord.

Flux	Étapes principales	Résultat attendu
Assistant intelligent	Construction du contexte, sélection des données utiles, appel IA ou fallback local, retour structuré.	Conseil contextualisé, plan d'action ou résumé de journée.
Import CSV	Téléversement, validation du fichier, lecture des lignes, création contrôlée des étudiants.	Réduction de la saisie manuelle et cohérence des comptes.
Notification	Déclenchement par action utilisateur ou agent, création en base, consultation REST ou temps réel.	Rappel visible par l'étudiant sans interrompre le parcours principal.
Tableau de bord	Agrégation des check-ins, objectifs, tâches, échéances et modes.	Vue synthétique de la situation académique et personnelle.

Table 15: Flux fonctionnels détaillés

Pour éviter la duplication de logique, les traitements communs sont centralisés dans les services. Par exemple, la préparation du contexte étudiant est utilisée par l'assistant, les notifications et certains indicateurs du tableau de bord. Ainsi, l'application garde une source de vérité cohérente pour les informations sensibles comme les échéances, les objectifs et les derniers check-ins.

3.4 Conception technique

3.4.1 Diagramme de code C4 – Niveau 4

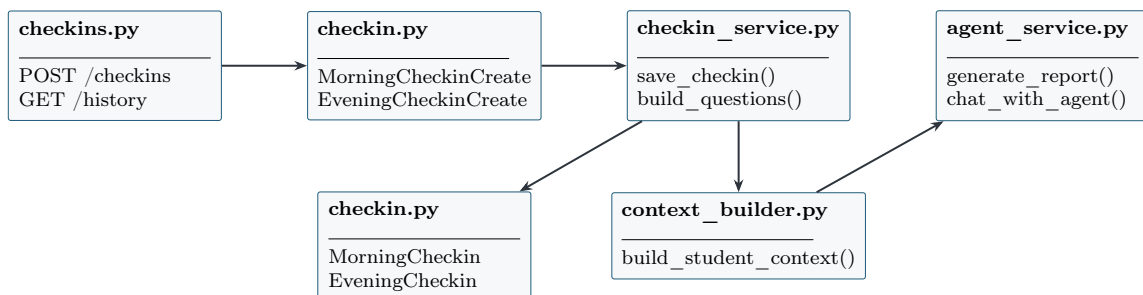


Figure 5: Diagramme de code C4 niveau 4 du module check-in

3.4.2 Structure des dossiers

```

mizan/
|-- mizan-backend/      # API FastAPI, models, schemas, services, tests
|-- mizan-frontend/     # Application web Next.js
|-- mizan-mobile-app/   # Application mobile Expo React Native
|-- infra/aws/          # Infrastructure Terraform AWS
  
```

```
|-- docker/          # Configuration Docker et nginx
|-- .github/workflows/ # CI/CD GitHub Actions
|-- rapport/         # Rapport LaTeX
```

3.4.3 Organisation des classes et interaction entre modules

Les classes ORM représentent les entités persistantes. Les schémas Pydantic assurent la validation. Les services orchestrent les interactions entre modèles, contexte et fournisseurs externes. Ce découpage reprend les principes MVC et service layer.

Principe	Application dans Mizan	Apport
Repository implicite	Les requêtes SQLAlchemy sont regroupées dans les services ou fonctions spécialisées.	Limite la dispersion des accès à la base.
Service layer	Les règles métier sont placées dans <code>services/</code> .	Facilite les tests et l'évolution des modules.
DTO / Schémas	Les entrées et sorties passent par Pydantic.	Réduit les erreurs de format et documente les contrats API.
Middleware de sécurité	Les dépendances FastAPI identifient l'utilisateur et son rôle.	Protège les routes sans répéter le même code.
Fallback applicatif	L'assistant peut répondre en mode local si le fournisseur IA est absent.	Améliore la disponibilité fonctionnelle.

Table 16: Principes de conception appliqués

L'interaction entre modules suit généralement le même enchaînement : une route reçoit la demande, un schéma valide les données, un service applique les règles, un modèle interagit avec PostgreSQL, puis une réponse normalisée est renvoyée. Ce chemin rend le comportement plus prévisible, surtout pour les fonctionnalités qui combinent plusieurs sources de données.

3.5 Modélisation UML

La modélisation UML décrit la structure interne du domaine métier et complète les vues C4. Les diagrammes C4 expliquent l'architecture par niveaux, tandis que l'UML précise les entités, leurs relations et certains échanges dynamiques. Dans Mizan, cette modélisation est utile pour relier les fonctionnalités aux données persistées : utilisateur, étudiant, classe, check-ins, objectifs, tâches, ressources, notifications et décisions de l'assistant.

Diagramme UML	Rôle dans le rapport	Information apportée
Cas d'utilisation	Synthétiser les interactions entre acteurs et système.	Fonctions accessibles à l'étudiant, à l'administrateur et à l'assistant IA.
Classes	Structurer le domaine persistant.	Entités principales, attributs importants et cardinalités.
Séquence	Montrer les échanges pendant un scénario.	Ordre des appels entre front-end, API, base, contexte et assistant.

Table 17: Rôle des diagrammes UML dans Mizan

3.5.1 Diagrammes de séquence

Les diagrammes de séquence complètent le diagramme de classes en décrivant l'ordre des échanges entre les acteurs, les interfaces, l'API, les services métier et la base de données. Ils permettent de vérifier que la conception technique reste cohérente avec les cas d'utilisation : chaque action utilisateur passe par une interface, une validation côté API, un traitement métier, puis une réponse exploitable.

Le scénario retenu est le check-in étudiant avec analyse intelligente, car il représente le coeur de l'expérience Mizan : l'étudiant saisit son état quotidien, le système enregistre les données, construit un contexte récent, puis prépare une réponse d'accompagnement exploitable. Ce flux montre à la fois l'interaction utilisateur, la validation côté API, la persistance des données et l'appel au service intelligent.

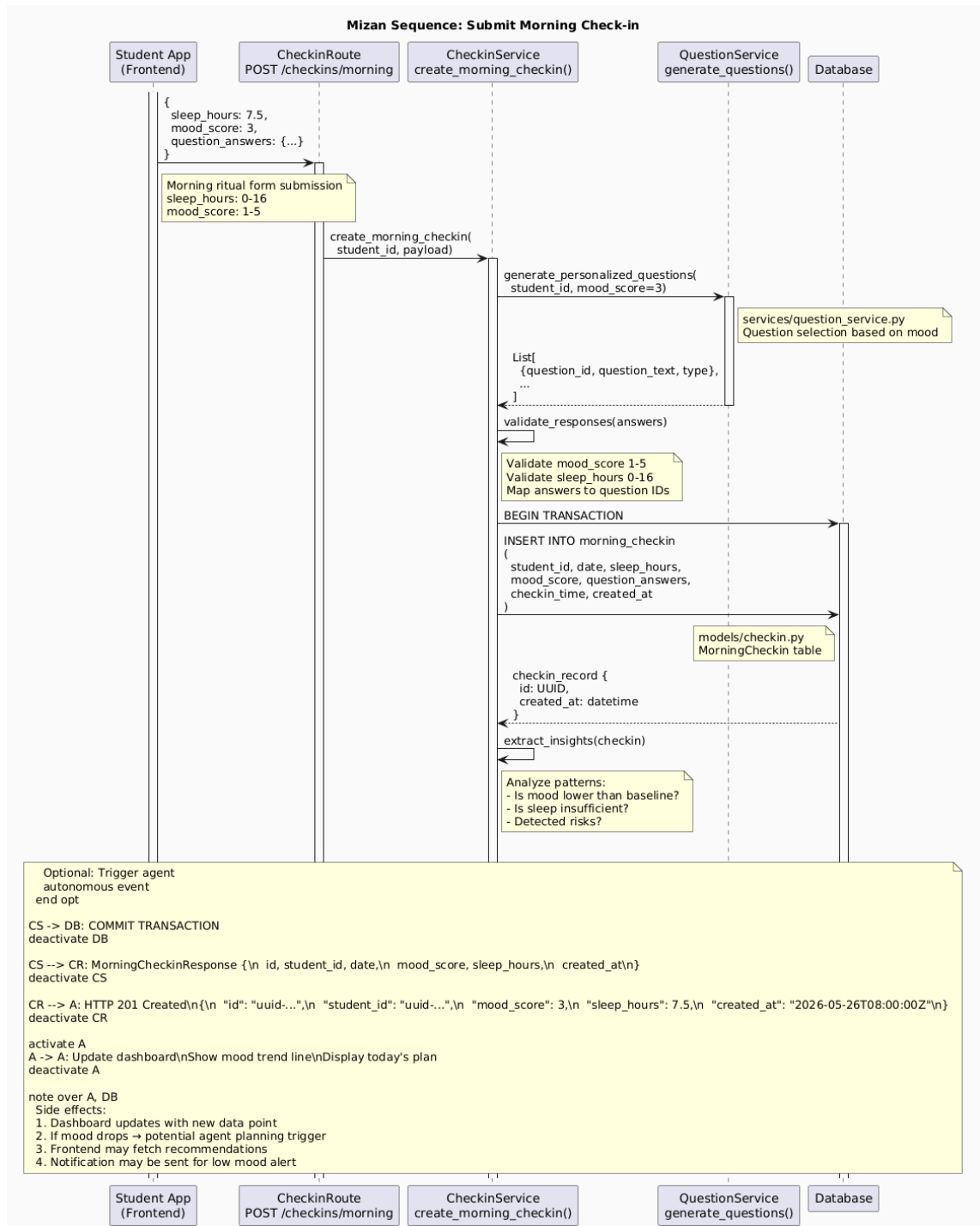


Figure 6: Diagramme de séquence du check-in avec analyse intelligente

Ce diagramme montre que le check-in n'est pas seulement un formulaire enregistré en base. Après la validation, les données saisies servent à construire un contexte étudiant. Ce contexte regroupe les informations récentes utiles, puis il est transmis à l'assistant afin de produire un résumé, un conseil ou un plan d'action. Si le fournisseur IA n'est pas disponible, le service peut appliquer un mode de secours local afin de conserver une réponse minimale pour l'utilisateur.

3.5.2 Diagramme de classes

Le diagramme de classes représente une vue simplifiée du modèle de données. Il montre que l'entité **User** porte l'identité et le rôle, tandis que **Student** décrit le profil académique rattaché à une classe. Les entités comme **Checkin**, **Goal**, **Task** et **ModeSession** appartiennent à l'étudiant et alimentent le suivi quotidien. Les entités **AgentRun**, **Notification** et **Resource** servent à transformer ce suivi en accompagnement exploitable.

3 Conception de la Solution

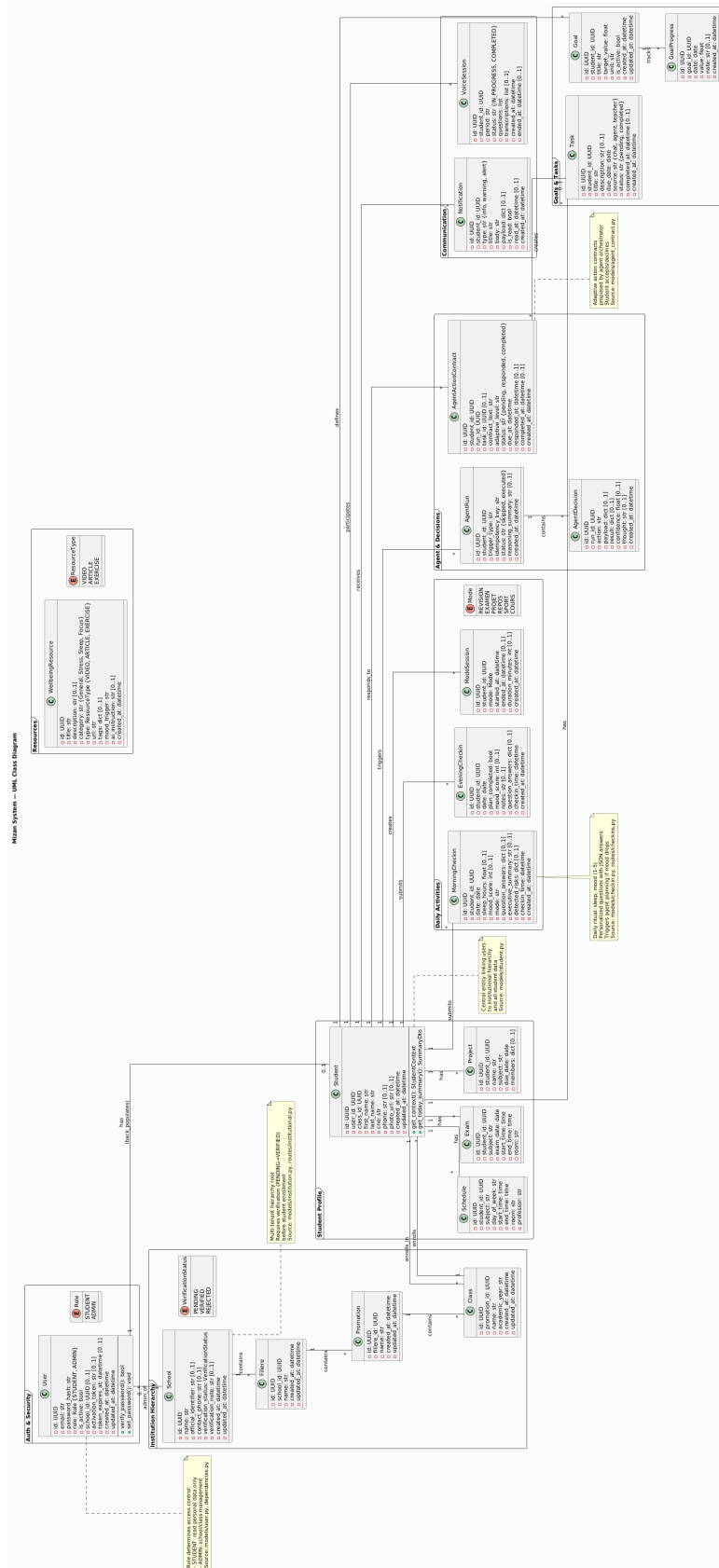


Figure 7: Diagramme de classes détaillé du domaine Mizan

Ce diagramme distingue deux groupes d'entités. Le premier groupe décrit l'identité et l'organisation académique : **User**, **Student**, **Class** et **School**. Il permet d'assurer le rattachement correct de chaque étudiant à une classe et de limiter les accès selon le rôle de l'utilisateur. Le deuxième groupe décrit le suivi quotidien : check-ins, objectifs, tâches, sessions de mode, ressources, notifications et exécutions de l'agent.

Zone du modèle	Rôle	Impact fonctionnel
Identité et rôles	Gérer les comptes, droits et profils.	Connexion, accès étudiant, accès administrateur.
Structure académique	Relier écoles, classes et étudiants.	Filtrage des données et gestion administrative.
Suivi personnel	Stocker check-ins, objectifs, tâches et modes.	Tableau de bord, historique et recommandations.
Accompagnement	Relier agent, ressources et notifications.	Conseils contextualisés et actions proposées.

Table 18: Lecture fonctionnelle du diagramme de classes

La relation entre **Student** et les entités de suivi est centrale : elle garantit que les informations personnelles restent attachées au bon profil. Les cardinalités de type 1..N montrent qu'un étudiant peut avoir plusieurs check-ins, objectifs, tâches ou notifications. Ce choix correspond à l'usage réel de Mizan, où les données s'accumulent au fil des jours afin de construire un historique exploitable.

3.5.3 Diagramme de déploiement

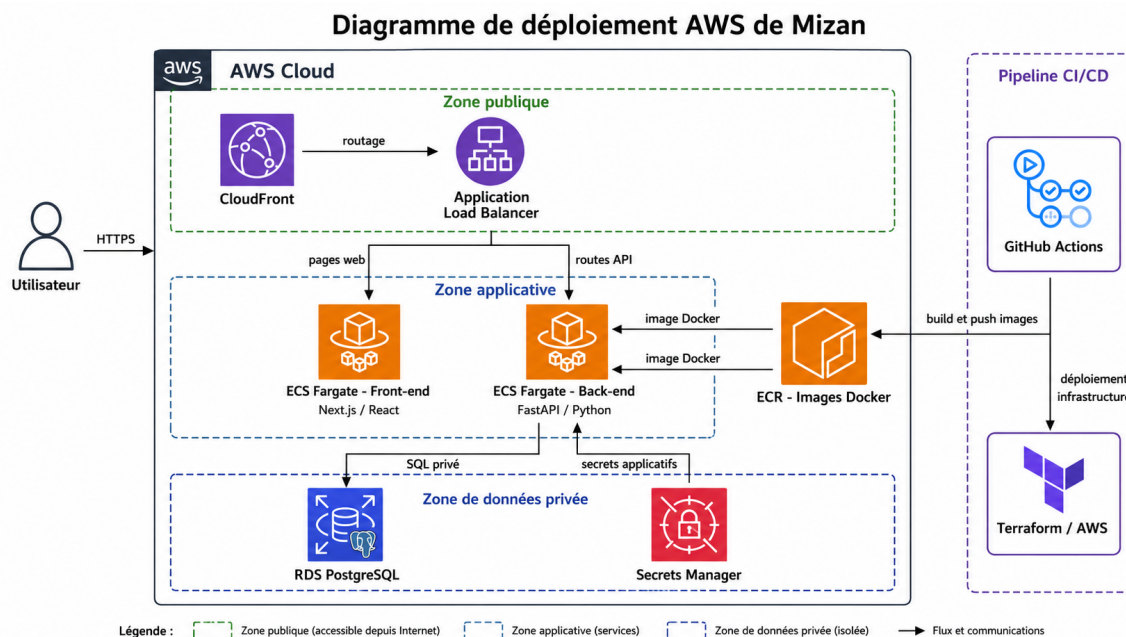


Figure 8: Diagramme de déploiement AWS

Le diagramme de déploiement présente l'architecture cible prévue pour rendre la solution accessible de manière structurée. CloudFront représente le point d'entrée public, tandis que l'Application Load Balancer oriente les requêtes vers le front-end ou vers l'API. Les services front-end et back-end sont isolés afin de pouvoir les faire évoluer séparément. La base PostgreSQL reste dans une zone privée, accessible uniquement par le back-end, ce qui réduit l'exposition des données.

Composant	Responsabilité	Justification
CloudFront	Point d'accès public et diffusion frontale.	Améliorer l'accès utilisateur et préparer le HTTPS.
Load Balancer	Routage vers les services applicatifs.	Séparer les routes web et API.
ECS Fargate front-end	Exécution de l'application web.	Déployer l'interface sans gérer de serveur manuel.
ECS Fargate back-end	Exécution de l'API FastAPI.	Isoler les règles métier et les accès base.
RDS PostgreSQL	Persistance relationnelle.	Centraliser les données critiques dans un stockage managé.
Secrets Manager	Gestion des secrets sensibles.	Éviter les clés et mots de passe dans le code.

Table 19: Rôle des composants de déploiement

Cette architecture cible reste cohérente avec les exigences non fonctionnelles : sécurité, maintenabilité, évolutivité et séparation des responsabilités. Elle permet aussi de préparer un passage progressif vers une exploitation plus complète, avec supervision, journaux structurés et contrôle des variables d'environnement.

3.6 Conception des interfaces

Logique générale

La conception des interfaces repose sur une séparation claire entre les usages étudiants et les usages administratifs. L'étudiant doit accéder rapidement aux informations qui l'aident dans sa journée : humeur, sommeil, priorités, objectifs, tâches, modes de travail, notifications et recommandations. L'administrateur, au contraire, a besoin d'écrans plus structurés pour gérer des listes, importer des données, consulter des classes et maintenir les contenus académiques.

Cette différence influence directement l'organisation visuelle. Les écrans étudiants privilégient les cartes de synthèse, les actions rapides et les formulaires courts. Les écrans administrateur privilégient les tableaux, les filtres, les boutons d'action et les formulaires de gestion. Le but n'est pas de multiplier les pages, mais de réduire l'effort mental pour chaque type d'utilisateur.

Parcours utilisateur

Le parcours étudiant commence par la connexion, puis le tableau de bord. L'étudiant peut réaliser un check-in, consulter ses tâches, suivre ses objectifs et demander de l'aide à l'assistant. Le parcours administrateur commence par la connexion à l'espace admin, puis la gestion des structures, étudiants et contenus.

Profil	Besoin principal	Réponse d'interface
Étudiant	Comprendre sa journée rapidement.	Dashboard synthétique, check-in court, objectifs visibles.
Étudiant en période chargée	Prioriser sans perdre du temps.	Rappels, échéances proches, assistant et plan d'action.
Administrateur d'école	Gérer classes et étudiants.	Tableaux, import CSV, actions de création et modification.
Administrateur global	Suivre les établissements.	Écrans de supervision et validation.

Table 20: Correspondance entre profils et interfaces

Ergonomie et navigation

L'ergonomie repose sur des pages dédiées, une navigation claire, des formulaires courts et des tableaux synthétiques. L'application mobile facilite les usages rapides, notamment les check-ins.

Les interfaces sont pensées selon la fréquence d'utilisation. Les actions quotidiennes, comme le check-in ou la consultation du tableau de bord, doivent être accessibles rapidement. Les actions administratives, plus structurées, privilégient les tableaux, formulaires et imports. Cette différence permet de proposer une expérience simple à l'étudiant tout en gardant une gestion complète pour l'administrateur.

Parcours	Écrans concernés	Point d'attention ergonomique
Connexion et activation	Login, activation, mot de passe.	Messages clairs et étapes courtes.
Check-in quotidien	Check-in matin, check-in soir, historique.	Formulaire rapide et feedback immédiat.
Suivi personnel	Dashboard, objectifs, tâches, modes.	Synthèse visuelle et priorité aux informations utiles.
Accompagnement IA	Chat, scénarios, contrats d'action.	Réponses lisibles et actions concrètes.
Administration	Classes, étudiants, contenus, ressources.	Tableaux structurés, filtres et imports.

Table 21: Parcours utilisateur et exigences d'interface

Organisation des écrans

L'organisation des écrans suit une logique de priorité. Les informations les plus fréquentes apparaissent en premier : état du jour, objectifs, tâches, échéances proches et notifications. Les informations plus détaillées, comme l'historique ou les ressources, restent accessibles par navigation. Cette hiérarchie évite de transformer le tableau de bord en page trop dense.

Écran	Contenu principal	Justification
Dashboard étudiant	Humeur, sommeil, tâches, objectifs, échéances.	Donner une vision immédiate de la journée.
Check-in	Questions courtes matin/soir.	Favoriser une saisie régulière.
Objectifs et tâches	Progression, statut, actions rapides.	Aider l'étudiant à suivre ses engagements.
Assistant	Conversation, recommandations, plans d'action.	Transformer les données en conseils exploitables.
Administration	Classes, étudiants, contenus, ressources.	Centraliser la gestion pédagogique.

Table 22: Organisation logique des écrans

Cohérence avec les cas d'utilisation

Chaque interface correspond à un cas d'utilisation identifié : check-in, suivi des objectifs, gestion institutionnelle, import, consultation des statistiques et dialogue avec l'assistant.

La cohérence est assurée par la correspondance entre acteurs, besoins et écrans. Le check-in répond au besoin de suivi quotidien. Le dashboard répond au besoin de synthèse. Les pages d'objectifs et de tâches répondent au besoin d'organisation. L'espace administrateur répond au besoin de gestion académique. Enfin, l'assistant répond au besoin d'accompagnement personnalisé. Cette logique évite d'ajouter des interfaces sans lien direct avec les besoins exprimés.

4 Réalisation et Implémentation

4.1 Environnement de développement

Élément	Valeur
Système d'exploitation	Linux pour le développement local.
IDE / éditeur	Éditeur de code moderne avec terminal intégré.
Versions principales	Python 3.12, Node.js 20, Next.js, React, Expo, PostgreSQL.
Configuration matérielle	Poste de développement standard et architecture cloud cible.

Table 23: Environnement de développement

4.2 Architecture réalisée

L'architecture réalisée respecte la séparation des composants présentée dans les diagrammes de conception : front-end web, application mobile, API FastAPI, base PostgreSQL et intégrations externes. La structure réelle du code sépare clairement les applications et facilite les tests indépendants.

Le back-end regroupe les routes HTTP, la configuration, la sécurité, les modèles SQLAlchemy, les schémas Pydantic et les services métier. Le front-end web regroupe les pages étudiant, les pages administrateur, les composants UI et la couche API. L'application mobile reprend les usages rapides, notamment l'accès au back-end depuis un téléphone ou un émulateur. Cette séparation permet de faire évoluer un composant sans bloquer les autres.

Composant	Responsabilité réalisée	Exemple de livrable
Back-end	API, règles métier, sécurité, persistance.	Routes <code>/api/v1</code> , services et modèles.
Front-end web	Interface étudiant et administration.	Pages dashboard, check-in, classes et ressources.
Mobile	Accès rapide au suivi étudiant.	Application Expo React Native.
Base de données	Stockage relationnel des entités.	Tables étudiants, check-ins, objectifs, notifications.

Composant	Responsabilité réalisée	Exemple de livrable
Infrastructure	Préparation du déploiement et automatisé.	Docker, Terraform et GitHub Actions.

Table 24: Correspondance entre conception et réalisation

4.3 Implémentation des fonctionnalités

Authentification et contrôle d'accès

L'authentification utilise des jetons JWT, des jetons de rafraîchissement et des dépendances FastAPI pour identifier l'utilisateur courant. Les rôles **ADMIN** et **STUDENT** protègent les routes sensibles.

Lorsqu'un utilisateur se connecte, l'API vérifie ses identifiants, génère un jeton d'accès et un jeton de rafraîchissement. Le front-end ajoute ensuite automatiquement le jeton d'accès aux requêtes. Si le jeton expire, la couche API tente un renouvellement avant de rediriger l'utilisateur vers la page de connexion.

Gestion des utilisateurs

La gestion des utilisateurs couvre l'activation, la connexion, la création des étudiants, l'import CSV et le rattachement aux classes.

L'import CSV répond à un besoin pratique : créer plusieurs étudiants sans saisie manuelle répétitive. Les données importées doivent être validées avant insertion afin d'éviter les comptes incomplets ou les rattachements à des classes inexistantes.

Gestion des données métier

Les données métier incluent écoles, filières, promotions, classes, étudiants, emplois du temps, examens, projets, check-ins, objectifs, tâches, modes, ressources et notifications.

Ces données sont organisées autour du profil étudiant. Les emplois du temps, examens et projets enrichissent le contexte académique. Les check-ins, objectifs, tâches et modes décrivent l'état personnel et l'organisation quotidienne. Les ressources et notifications servent à transformer ce contexte en accompagnement concret.

Opérations CRUD

Les opérations CRUD sont exposées par des routes REST. Les schémas Pydantic valident les entrées et les services appliquent les règles métier avant la persistance.

Cette approche limite les responsabilités de chaque couche. Les routes reçoivent la requête, les schémas contrôlent la forme des données, les services appliquent les règles et les modèles assurent la persistance. Elle rend le code plus testable et plus lisible.

Export et notifications

Le système de notifications permet la consultation REST et la livraison temps réel pour les rappels et actions autonomes.

Les notifications sont utiles pour signaler un rappel de check-in, une recommandation de pause, une échéance ou une action proposée par l'assistant. Elles doivent rester pertinentes afin de ne pas créer de surcharge supplémentaire pour l'étudiant.

4.4 Intégration avec la base de données

Connexion et ORM

La connexion à PostgreSQL est gérée par SQLAlchemy en mode asynchrone. Alembic versionne les migrations afin de garder une évolution contrôlée du schéma.

Requêtes principales

Les requêtes principales concernent la récupération du contexte étudiant, l'historique des check-ins, les objectifs actifs, les échéances académiques et les indicateurs de tableau de bord.

Donnée consultée	Utilisation	Module consommateur
Historique des check-ins	Suivre l'évolution de l'humeur et du sommeil.	Dashboard, analytics, assistant.
Examens et projets	Identifier les échéances proches.	Assistant, notifications, dashboard.
Objectifs et tâches	Mesurer la progression quotidienne.	Dashboard, agent, pages objectifs.
Modes de travail	Comprendre la répartition du temps.	Historique, statistiques.
Ressources	Proposer un soutien adapté.	Assistant, notifications.

Table 25: Utilisation des données principales

Sécurité des accès

Les accès sont sécurisés par rôles, par validation de l'utilisateur courant et par scoping des administrateurs selon l'établissement.

Cette logique réduit les risques d'accès croisé entre établissements. Un administrateur ne doit pas manipuler les données d'une école qui ne lui appartient pas. De même, un étudiant ne doit consulter que ses propres informations.

4.5 Sécurité et contrôle d'accès

La sécurité du projet repose sur plusieurs niveaux :

- authentification par jetons JWT ;
- mots de passe hachés côté serveur ;
- contrôle du rôle `ADMIN` ou `STUDENT` selon les routes ;

- validation stricte des données avec Pydantic ;
- limites de taille pour les fichiers image, audio et CSV ;
- rejet des secrets faibles en production ;
- CORS restreint en environnement de production.

4.6 Plan de validation et qualité

Le plan de validation du projet repose sur une combinaison de vérifications automatiques et de scénarios fonctionnels. Les tests automatiques permettent de détecter rapidement les régressions dans le back-end, tandis que les commandes de lint et de build vérifient la cohérence des applications front-end et mobile. Cette démarche est importante car Mizan comporte plusieurs interfaces qui consomment la même API.

Zone validée	Vérification	Objectif
Back-end	Compilation Python et tests Pytest.	Vérifier les services, contrats et règles métier.
Front-end web	Lint et build Next.js.	Détecter les erreurs TypeScript, React et routage.
Mobile	Typecheck TypeScript.	Vérifier la cohérence des types et appels API.
Déploiement	Tests de readiness et configuration production.	Refuser les secrets faibles et paramètres incomplets.
Sécurité fichiers	Validation des extensions, types MIME et tailles.	Éviter les fichiers dangereux ou trop volumineux.
Assistant	Tests du fallback et de l'orchestration.	Maintenir une réponse utile même sans fournisseur externe.

Table 26: Stratégie de tests et validation

```
# Validation back-end
cd mizan-backend
python -m compileall -q app main.py tests
python -m pytest

# Validation front-end web
cd ../mizan-frontend
npm run lint
npm run build

# Validation mobile
cd ../mizan-mobile-app
npm run typecheck
```

Les scénarios fonctionnels à vérifier couvrent le parcours minimal : activation ou connexion d'un étudiant, réalisation d'un check-in, consultation du tableau de bord, création d'un objectif, échange avec l'assistant, puis consultation d'une notification. Côté administration, la validation concerne la création d'une structure institutionnelle, l'ajout ou l'import d'étudiants, l'ajout d'un examen et la consultation d'indicateurs.

Scénario validé	Critère de réussite	Risque couvert
Connexion étudiant	Jeton reçu et redirection vers le dashboard.	Blocage d'accès ou erreur d'authentification.
Check-in matin	Données sauvegardées et historique mis à jour.	Perte de données quotidiennes.
Assistant	Réponse générée ou fallback local affiché.	Dépendance totale à un service externe.
Import étudiants	Lignes valides créées et erreurs signalées.	Données administratives incohérentes.
Contrôle des rôles	Route admin inaccessible à un étudiant.	Accès non autorisé.

Table 27: Matrice de validation fonctionnelle

4.7 Préparation du déploiement et exploitation

La préparation du déploiement est organisée autour de Docker, Terraform et GitHub Actions. Docker fournit un environnement reproductible pour le front-end et le back-end. Terraform décrit l'infrastructure AWS cible, notamment le réseau, les services ECS, le registre ECR, la base RDS, CloudFront et l'Application Load Balancer. GitHub Actions prépare les étapes de vérification, construction et déploiement.

Élément d'exploitation	Rôle	Exemple de configuration
Variables d'environnement	Adapter l'application à chaque environnement.	DATABASE_URL, JWT_SECRET, CORS_ORIGINS.
Secrets applicatifs	Protéger clés et mots de passe.	Secrets AWS ou variables GitHub Actions.
Images Docker	Emballer les applications.	Images back-end et front-end prévues pour ECR.
Migrations Alembic	Faire évoluer le schéma SQL.	Exécution contrôlée avant mise en production.
Endpoint de santé	Vérifier la disponibilité du service.	Route de health check prévue pour l'infrastructure.

Table 28: Éléments de déploiement et exploitation

En exploitation, les points de vigilance principaux sont la surveillance des erreurs API, la disponibilité de la base de données, la validité des secrets, la taille des fichiers envoyés et la stabilité des appels vers les fournisseurs externes. Une amélioration future consiste à ajouter une observabilité plus complète : journaux structurés, métriques applicatives, alertes et suivi des temps de réponse par route.

4.8 Difficultés rencontrées

Problèmes techniques et d'intégration

Les principales difficultés concernent l'intégration de l'IA avec un mode de secours, la cohérence entre le web et le mobile, la gestion des secrets, la validation des fichiers et la préparation du déploiement cloud.

Une autre difficulté a été la construction d'un contexte suffisamment riche pour l'assistant sans rendre le système trop complexe. L'assistant doit recevoir les informations utiles, mais il ne doit pas dépendre d'un nombre excessif de données obligatoires. Il fallait donc prévoir un fonctionnement progressif : plus le contexte est riche, plus la réponse peut être personnalisée, mais l'application doit rester utilisable avec un contexte minimal.

Solutions adoptées

Les solutions adoptées sont la modularisation des services, la validation stricte des paramètres, l'utilisation d'un fallback local pour l'IA, la centralisation de la couche API front-end, l'organisation des tests et l'infrastructure décrite par Terraform.

La solution adoptée consiste aussi à séparer les responsabilités. Le service de contexte prépare les données utiles, le service agent produit la réponse, le service notification se charge de la diffusion et les routes gardent un rôle d'orchestration HTTP. Cette organisation limite le couplage et facilite les évolutions futures.

Conclusion Générale

Bilan du projet

Mizan constitue une solution complète pour accompagner les étudiants dans leur organisation et leur bien-être quotidien. Le projet associe des fonctionnalités académiques, personnelles et intelligentes dans une architecture moderne : FastAPI, Next.js, Expo React Native, PostgreSQL, Docker, Terraform et architecture AWS cible.

Le travail réalisé a permis de passer d'un besoin général, celui d'aider l'étudiant à mieux gérer son quotidien, à une solution structurée composée de modules cohérents. L'application ne se limite pas à une simple liste de tâches : elle relie le contexte académique, les check-ins, les objectifs, les notifications et l'assistant intelligent. Cette combinaison donne au projet une valeur particulière, car elle aborde à la fois l'organisation, le suivi personnel et l'accompagnement.

Réponse à la problématique

La solution répond à la problématique en centralisant les informations académiques et les signaux de bien-être dans une même plateforme. Elle permet de produire des tableaux de

bord, des recommandations et des notifications contextualisées.

La réponse proposée repose sur trois axes. Le premier est la centralisation : l'étudiant retrouve ses informations importantes dans un espace unique. Le deuxième est la contextualisation : les recommandations prennent en compte les objectifs, les échéances, les check-ins et les données disponibles. Le troisième est la séparation des rôles : l'étudiant agit sur son suivi personnel, tandis que l'administrateur gère les données académiques nécessaires au bon fonctionnement de la plateforme.

Objectifs atteints

Les objectifs atteints concernent l'authentification, la gestion institutionnelle, les profils étudiants, les check-ins, les objectifs, les modes, l'assistant, les ressources et les notifications.

Au-delà de la réalisation fonctionnelle, le projet a permis de mettre en place une logique de traçabilité entre besoins, conception et implémentation. Chaque module principal répond à un besoin identifié : les check-ins pour le suivi quotidien, les objectifs et tâches pour l'organisation, les contenus académiques pour le contexte, l'assistant pour l'accompagnement et les tableaux de bord pour la synthèse.

Limites du travail réalisé

Certaines fonctionnalités IA dépendent de clés externes et de la disponibilité du fournisseur. L'application mobile peut encore être enrichie pour couvrir toutes les fonctions du web. Les tests end-to-end utilisateur peuvent également être renforcés.

Une autre limite concerne l'évaluation réelle de l'impact sur les étudiants. La plateforme fournit des outils de suivi et d'accompagnement, mais son efficacité doit être mesurée sur une période d'utilisation plus longue, avec des retours utilisateurs et des indicateurs qualitatifs. La qualité des recommandations dépend également de la régularité des check-ins et de la richesse des données académiques renseignées.

Il faut également souligner que les données de bien-être exigent une attention particulière. Même si la solution prévoit des mécanismes de sécurité, une version plus avancée devrait formaliser davantage le consentement, la durée de conservation des données, l'export et la suppression des informations personnelles. Ces éléments sont importants pour renforcer la confiance des utilisateurs.

Perspectives d'amélioration

Les perspectives prioritaires sont l'ajout de tests end-to-end, l'amélioration des tableaux de bord, la parité fonctionnelle mobile, les notifications natives et une politique explicite de consentement et d'export des données.

À moyen terme, Mizan pourrait intégrer une analyse plus avancée des tendances, un calendrier partagé, des exports PDF, des recommandations personnalisées par période académique et une meilleure prise en charge du travail en groupe. Sur le plan technique, l'ajout d'une observabilité complète et d'un pipeline de déploiement multi-environnements renforcerait la fiabilité de la solution.

Perspective	Description	Apport attendu
Tests end-to-end	Automatiser les parcours étudiant et administrateur.	Réduire les régressions fonctionnelles.
Mobile complet	Couvrir davantage de fonctionnalités web côté mobile.	Améliorer l'usage quotidien.
Consentement et données	Ajouter des règles explicites d'export, suppression et conservation.	Renforcer la confiance et la conformité.
Analytique avancée	Étudier les tendances par période, classe ou type d'échéance.	Aider à repérer les périodes de surcharge.
Notifications natives	Ajouter des rappels mobiles mieux intégrés.	Améliorer la régularité des check-ins.
Observabilité	Ajouter journaux structurés, métriques et alertes.	Faciliter le suivi technique.

Table 29: Perspectives d'amélioration de Mizan

Ces perspectives ne changent pas l'objectif initial du projet ; elles l'approfondissent. La priorité reste de conserver une solution utile et simple pour l'étudiant. Les améliorations futures doivent donc être évaluées selon leur capacité à réduire la charge mentale, clarifier les priorités et rendre les informations plus exploitables.

Apport personnel et professionnel

Ce projet a permis de consolider des compétences en analyse des besoins, conception UML, architecture C4, développement back-end, développement front-end, développement mobile, sécurité applicative, organisation des tests et préparation du déploiement cloud.

Sur le plan personnel, Mizan a permis de travailler sur un sujet qui combine technique et utilité concrète. La réalisation a demandé une vision globale : comprendre le besoin, structurer les données, concevoir les interfaces, organiser les modules et préparer une architecture évolutive. Sur le plan professionnel, ce projet a renforcé la pratique du développement full-stack, de la documentation technique et de la modélisation logicielle.

Annexe A – Glossaire Technique

Terme	Définition
Back-end	Partie serveur responsable de l'API, des règles métier et de la base de données.
Front-end	Interface utilisée par l'utilisateur depuis le navigateur.
Check-in	Saisie régulière permettant de suivre l'humeur, le sommeil et le bilan de l'étudiant.
Conteneur	Unité d'exécution Docker embarquant une application et ses dépendances.
Migration	Script de versionnement permettant de faire évoluer le schéma de base de données.

Table 30: Glossaire technique

Bibliographie / Webographie

- Documentation FastAPI : <https://fastapi.tiangolo.com/>
- Documentation Next.js : <https://nextjs.org/docs>
- Documentation Expo : <https://docs.expo.dev/>
- Documentation SQLAlchemy : <https://docs.sqlalchemy.org/>
- Documentation Terraform AWS Provider.
- Documentation locale du projet Mizan : README.md, mizan-backend/README.md, mizan-mobile-app/README.md, DEPLOYMENT_README.md.